

LAB MANUAL

Week 1: Python Refresher, NumPy, Pandas & Matplotlib

A 6-Day Practical Lab Guide for Data Science / Machine Learning Foundations

Duration: 6 Days (2–3 hours/day)

Prerequisites: Basic computer literacy; Python 3.x installed (or Google Colab / Jupyter Notebook)

Tools Required: Python 3.x, pip, NumPy, Pandas, Matplotlib, Jupyter Notebook / VS Code / any IDE

Includes: Daily topics, syntax references, worked examples, practice tasks, and a 25-question graded lab exercise

Course: Machine Learning Fall 2026 — Lab 1 (Week 1)

Week Overview

This lab manual guides students from core Python fundamentals through array-based scientific computing with NumPy, tabular data handling with Pandas, and data visualization with Matplotlib — the four building blocks used every week for the rest of the course. Each day builds on the previous day's concepts and ends with hands-on practice tasks.

Day	Focus Area
Day 1	Python Refresher: Variables, Data Types, Operators & Control Flow
Day 2	Python Refresher: Functions, Data Structures & Comprehensions
Day 3	NumPy Fundamentals: Arrays, Indexing, Slicing & Operations
Day 4	Pandas I: Series, DataFrames, Reading Data & Selection
Day 5	Pandas II: Cleaning, Grouping & Merging
Day 6	Matplotlib: Plotting & Mini End-to-End Task

Day 1: Python Refresher — Variables, Data Types, Operators & Control Flow

Learning Objectives

- Recall Python syntax rules, indentation, and program structure
- Declare variables, use core data types, and cast between them
- Apply arithmetic, comparison, logical, and bitwise operators
- Write conditional logic (if / elif / else) and for / while loops
- Use f-strings for formatted output

1.1 Python Basics, Variables & Data Types

Python is dynamically typed — a variable's type is inferred from the assigned value and can change at runtime. Indentation (not braces) defines code blocks.

Syntax Reference

Syntax	Description
<code>variable = value</code>	Assigns a value to a variable
<code>type(x)</code>	Returns the data type of x

<code>int(x) / float(x) / str(x) / bool(x)</code>	Type casting functions
<code>print(x)</code>	Prints value(s) to console
<code>a, b = 1, 2</code>	Multiple assignment in one line

Example

```
name = "Alice"      # str
age = 21            # int
gpa = float("3.85") # cast str -> float
print(type(name), type(age), type(gpa))
```

1.2 Operators

Syntax Reference

Syntax	Description
<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>%</code> <code>**</code>	Arithmetic: add, sub, mul, div, floor-div, modulo, power
<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>	Comparison operators, return bool
<code>and</code> <code>or</code> <code>not</code>	Logical operators
<code>&</code> <code> </code> <code>^</code> <code>~</code> <code><<</code> <code>>></code>	Bitwise operators
<code>is</code> <code>is not</code> , <code>in</code> <code>not in</code>	Identity and membership tests

Example

```
a, b = 17, 5
print(a // b, a % b, a ** 2) # 3 2 289
print(3 in [1, 2, 3])      # True
```

1.3 Conditional Statements & Loops

Syntax Reference

Syntax	Description
<code>if</code> <code>elif</code> <code>else</code>	Branching logic
<code>x if cond else y</code>	Ternary expression
<code>for var in iterable:</code>	Iterates over a sequence
<code>range(start, stop, step)</code>	Generates a sequence of numbers
<code>while cond:</code>	Repeats while condition is True
<code>break</code> <code>continue</code> <code>pass</code>	Loop control statements

Example

```
for i in range(1, 6):
    if i == 4:
        continue
    print(i)          # 1 2 3 5
```

Practice Tasks

- Take two numbers as input and print their sum, difference, product, quotient, floor division, and modulus.
- Convert a temperature given in Fahrenheit (string input) to Celsius using type casting and an f-string for output.
- Print all prime numbers between 1 and 100 using nested loops and break.
- Write a program using a bitwise operator to check whether a given integer is even or odd (hint: $n \& 1$).

Day 2: Python Refresher — Functions, Data Structures & Comprehensions

Learning Objectives

- Define functions with positional, default, `*args`, and `**kwargs` parameters
- Write lambda functions
- Create and manipulate lists, tuples, dictionaries, and sets
- Apply slicing and common built-in methods
- Write list and dictionary comprehensions

2.1 Functions

Syntax Reference

Syntax	Description
<code>def func(params): / return value</code>	Defines and returns from a function
<code>def func(a, b=10):</code>	Default parameter value
<code>def func(*args): / def func(**kwargs):</code>	Variable-length positional / keyword args
<code>lambda x: expr</code>	Anonymous, single-expression function

Example

```
def greet(name, msg="Hello"):
    return f"{msg}, {name}!"

def total(*nums):
    return sum(nums)

square = lambda x: x ** 2
```

2.2 Lists, Tuples, Sets & Dictionaries

Syntax Reference

Syntax	Description
<code>lst[start:stop:step]</code>	Slicing a list
<code>lst.append(x) / .pop(i) / .sort()</code>	Common list methods (mutable, ordered)
<code>t = (1, 2, 3)</code>	Tuple: ordered, immutable
<code>s1 s2, s1 & s2, s1 - s2</code>	Set union, intersection, difference
<code>d[key] = value, d.get(key, default)</code>	Dictionary assignment / safe lookup

<code>d.keys()</code> / <code>.values()</code> / <code>.items()</code>	Views of keys, values, or pairs
--	---------------------------------

2.3 Comprehensions

Syntax Reference

Syntax	Description
<code>[expr for x in iterable]</code>	List comprehension
<code>[expr for x in it if cond]</code>	Filtered list comprehension
<code>{k: v for k, v in ...}</code>	Dictionary comprehension

Example

```
evens = [x for x in range(20) if x % 2 == 0]
word_len = {w: len(w) for w in ["cat", "elephant", "dog"]}
print(word_len)  # {'cat': 3, 'elephant': 8, 'dog': 3}
```

Practice Tasks

- Write a function `stats(*nums)` that returns a tuple of (min, max, average) using `*args`.
- Given a list of numbers, use list comprehension to create a list of squares of the even numbers only.
- Given two sets of student names enrolled in two courses, find students enrolled in both, and in only one.
- Write a function that removes duplicate elements from a list while preserving order.
- Create a dictionary comprehension mapping each number from 1–10 to its cube.

Day 3: NumPy Fundamentals — Arrays, Indexing, Slicing & Operations

Learning Objectives

- Create arrays using `array`, `zeros`, `ones`, `arange`, `linspace`, `eye`, `random`
- Inspect array attributes: `shape`, `dtype`, `ndim`, `size`
- Index, slice, and reshape 1D/2D arrays
- Perform element-wise operations, broadcasting, and aggregate/statistical functions
- Apply boolean masking and fancy indexing

3.1 Creating Arrays & Attributes

Syntax Reference

Syntax	Description
<code>np.array(list)</code>	Creates an array from a Python list/tuple
<code>np.zeros((r,c))</code> / <code>np.ones((r,c))</code> / <code>np.full((r,c), val)</code>	Arrays of zeros / ones / a constant
<code>np.arange(start, stop, step)</code> / <code>np.linspace(start, stop, n)</code>	Evenly spaced values
<code>np.eye(n)</code>	$n \times n$ identity matrix

<code>np.random.randint(low, high, size)</code>	Array of random integers
<code>arr.shape / .ndim / .size / .dtype</code>	Array attributes

Example

```
import numpy as np
arr = np.array([[1, 2, 3], [4, 5, 6]])
print(arr.shape, arr.ndim, arr.size, arr.dtype)    # (2,3) 2 6 int64
```

3.2 Indexing, Slicing & Reshaping

Syntax Reference

Syntax	Description
<code>arr[i, j]</code>	Access element at row i, column j (2D)
<code>arr[r1:r2, c1:c2]</code>	Slice rows and columns of a 2D array
<code>arr.reshape(r, c) / .flatten() / .T</code>	Reshape, flatten, or transpose

3.3 Operations, Broadcasting & Aggregation

Syntax Reference

Syntax	Description
<code>arr + / - / * / scalar or array</code>	Element-wise operations; broadcasting stretches size-1 dims
<code>np.sum / np.mean / np.std / np.max / np.argmax</code>	Aggregate & statistical functions
<code>axis=0 / axis=1</code>	Column-wise / row-wise aggregation
<code>arr[arr > n]</code>	Boolean masking
<code>np.where(cond, a, b)</code>	Element-wise conditional selection
<code>np.vstack([a,b]) / np.hstack([a,b])</code>	Stack arrays vertically / horizontally

Example

```
m = np.array([[1, 2, 3], [4, 5, 6]])
print(m.sum(axis=0))    # [5 7 9] column sums
print(m[m > 3])         # [4 5 6]
```

Practice Tasks

- Create a 1D array of the first 15 natural numbers; print its shape, size, and dtype.
- Generate a 5x5 array of random integers between 10 and 50; print its max, min, and mean.
- Given a 1D array of 20 elements, reshape into a 4x5 matrix and extract the 2nd and 3rd columns.
- Given a 1D array of 15 exam scores, use boolean masking to find scores above the mean.
- Use `np.where()` to replace negative values in an array with 0 and positive values with 1.

Day 4: Pandas I — Series, DataFrames, Reading Data & Selection

Learning Objectives

- Explain the relationship between NumPy arrays, Series, and DataFrames
- Create a DataFrame from a dict/list and from a CSV file
- Inspect a DataFrame: shape, dtypes, head/tail, info, describe
- Select data with column selection, .loc, and .iloc
- Filter rows using boolean conditions

4.1 Series & DataFrames

A Pandas Series is a labeled 1D array (built on NumPy); a DataFrame is a table of Series sharing a common index — think of it as the in-Python equivalent of an Excel sheet or SQL table.

Syntax Reference

Syntax	Description
<code>import pandas as pd</code>	Standard import convention
<code>pd.Series(list)</code>	Creates a labeled 1D array
<code>pd.DataFrame(dict_or_list)</code>	Creates a table from a dict of lists or list of dicts
<code>pd.read_csv(path)</code>	Loads a CSV file into a DataFrame
<code>df.head(n)</code> / <code>df.tail(n)</code>	First / last n rows
<code>df.shape</code> / <code>df.columns</code> / <code>df.dtypes</code>	Dimensions, column names, column types
<code>df.info()</code> / <code>df.describe()</code>	Structure summary / numeric summary statistics

Example

```
import pandas as pd
data = {"name": ["Ali", "Sara", "Omar"],
        "marks": [78, 92, 65]}
df = pd.DataFrame(data)
print(df.head())
print(df.describe())
```

4.2 Selecting & Filtering Data

Syntax Reference

Syntax	Description
<code>df["col"]</code> / <code>df[["col1", "col2"]]</code>	Select one column (Series) / multiple columns (DataFrame)
<code>df.loc[row_label, col_label]</code>	Label-based selection
<code>df.iloc[row_pos, col_pos]</code>	Position-based (integer) selection
<code>df[df["col"] > n]</code>	Boolean filtering of rows
<code>df.sort_values("col", ascending=False)</code>	Sort rows by a column

Example

```
top = df[df["marks"] > 70]
print(top.sort_values("marks", ascending=False))
print(df.loc[0, "name"])      # Ali
print(df.iloc[1, 1])         # 92
```

Practice Tasks

- Create a DataFrame of 6 students with columns name, subject, and marks; print df.info() and df.describe().
- Load any small CSV (or the one provided by your instructor) and print its shape, column names, and first 5 rows.
- Using boolean filtering, select all students who scored above 80.
- Using .loc and .iloc, retrieve the same row two different ways and confirm the results match.

Day 5: Pandas II — Cleaning, Grouping & Merging

Learning Objectives

- Detect and handle missing values (isna, fillna, dropna)
- Add, rename, and drop columns
- Group data and compute aggregate statistics with groupby
- Merge/join two DataFrames

5.1 Cleaning Data

Syntax Reference

Syntax	Description
df.isna().sum()	Count missing values per column
df.fillna(value)	Replace missing values
df.dropna()	Drop rows containing missing values
df.rename(columns={"old": "new"})	Rename columns
df.drop(columns=["col"])	Drop a column
df["new_col"] = expr	Add a computed column

Example

```
df["marks"] = df["marks"].fillna(df["marks"].mean())
df["grade"] = df["marks"].apply(lambda m: "A" if m >= 80 else "B")
```

5.2 Grouping & Merging

Syntax Reference

Syntax	Description
df.groupby("col").agg(...)	Group rows and aggregate (mean, sum, count, etc.)
df.groupby("col")["x"].mean()	Mean of column x within each group
pd.merge(df1, df2, on="key")	Joins two DataFrames on a common column
pd.concat([df1, df2])	Stacks DataFrames row-wise or column-wise

Example

```
avg_by_subject = df.groupby("subject")["marks"].mean()
print(avg_by_subject)
```

Practice Tasks

- Given a DataFrame with some missing marks, count missing values per column, then fill them with the column mean.
- Add a 'pass_fail' column derived from 'marks' using a lambda inside .apply().
- Group a students DataFrame by 'subject' and compute mean, min, and max marks per subject.
- Given two small DataFrames (students, and their attendance), merge them on a common student ID column.

Day 6: Matplotlib — Plotting & Mini End-to-End Task

Learning Objectives

- Create line, bar, scatter, and histogram plots
- Label axes, add titles and legends, and use subplots
- Plot directly from a Pandas DataFrame/Series
- Complete a short pipeline: load → clean → aggregate → visualize

6.1 Basic Plots

Syntax Reference

Syntax	Description
<code>import matplotlib.pyplot as plt</code>	Standard import convention
<code>plt.plot(x, y)</code>	Line plot
<code>plt.bar(x, y) / plt.scatter(x, y)</code>	Bar plot / scatter plot
<code>plt.hist(data, bins=n)</code>	Histogram
<code>plt.xlabel() / plt.ylabel() / plt.title() / plt.legend()</code>	Labeling
<code>plt.show()</code>	Renders the figure

Example

```
import matplotlib.pyplot as plt
plt.bar(df["name"], df["marks"], color="steelblue")
plt.xlabel("Student"); plt.ylabel("Marks"); plt.title("Marks by Student")
plt.show()
```

6.2 Subplots & Plotting from Pandas

Syntax Reference

Syntax	Description
<code>fig, axes = plt.subplots(rows, cols)</code>	Creates a grid of subplots
<code>axes[0].plot(...)</code>	Plots into a specific subplot
<code>df["col"].plot(kind="hist")</code>	Quick plotting directly from a DataFrame/Series


```
plt.savefig("file.png")
```

Saves the figure to disk

6.3 Mini End-to-End Task (guided, ungraded warm-up)

Work through this as a group before attempting the graded exercise — it chains everything from Days 3–6:

- Load a small CSV with `pd.read_csv()` (e.g. a class marks sheet provided by your instructor).
- Clean it: check for missing values with `isna()`, fill or drop as appropriate.
- Compute a NumPy/Pandas summary: mean, std, and top scorer per subject using `groupby`.
- Visualize it: one bar chart of average marks per subject, and one histogram of overall marks distribution.
- Write one sentence per chart describing what it shows.

Practice Tasks

- Plot a line chart of 10 days of temperature readings with proper axis labels and a title.
- Create a 1x2 subplot: a bar chart of category counts on the left, a histogram of a numeric column on the right.
- Using `df["col"].plot(kind="box")`, generate a boxplot directly from a DataFrame column and describe what it shows.

Weekly Lab Exercise (Graded Assessment)

Instructions

- Attempt all 25 questions in a single Jupyter Notebook / .py file named `lastname_week1_lab.ipynb`.
- Q1–8 test core Python; Q9–17 test NumPy; Q18–22 test Pandas; Q23–25 test Matplotlib.
- Show your code AND the output for each question. Add a short comment above each solution explaining your approach.
- Total Marks: 50 (2 marks each). Submit before the next lab session.

Section A — Python (Q1–Q8)

- Write a program that swaps the values of two variables without using a third variable.
- Write a function `is_prime(n)` that returns `True` if `n` is a prime number, otherwise `False`.
- Write a program to print the Fibonacci sequence up to `n` terms using a loop.
- Write a function that takes a list of numbers and returns a new list with duplicates removed, preserving order.
- Write a function `multiply(*args)` that returns the product of any number of arguments using `*args`.
- Write a dictionary comprehension mapping each character in a string to its frequency.
- Given a list of dictionaries representing employees (name, department, salary), find the employee with the highest salary.
- Write a lambda function combined with `filter()` to extract all odd numbers from a list.

Section B — NumPy (Q9–Q17)

- Create a 1D NumPy array of integers from 1 to 30 and reshape it into a 5x6 matrix.
- Create a 6x6 identity matrix and replace its diagonal with the values `[1,2,3,4,5,6]`.
- Given a NumPy array of 25 random integers between 1 and 100, find the sum, mean, and standard deviation.
- Given a 2D array of shape (4,4), extract the diagonal elements using `np.diag()` and compute their sum.
- Create two arrays of shape (3,3) and demonstrate the difference between element-wise multiplication (`*`) and matrix multiplication (`@`).
- Given an array of daily temperatures for a month, use boolean masking to find and count days above 35°C.
- Write NumPy code to normalize an array to the range 0–1 using `(x - min) / (max - min)`.
- Given a 2D array of shape (5,3) (5 students, 3 subjects), compute total and average marks per student using axis-based aggregation.
- Use `np.where()` to replace all even numbers in an array with -1, keeping odd numbers unchanged.

Section C — Pandas (Q18–Q22)

- Create a DataFrame of 8 students with columns name, section, and marks; print `df.describe()`.
- Load a provided CSV, report how many missing values exist per column, and fill numeric missing values with the column mean.
- Using `.loc` and boolean filtering, select all rows where marks are below 50 and print only the name and marks columns.
- Group the students DataFrame by 'section' and compute the mean and max marks per section.
- Given two DataFrames — students and their attendance — merge them on student ID and report students with attendance below 75%.

Section D — Matplotlib (Q23–Q25)

- Plot a bar chart of average marks per section (from Q21's groupby result) with labeled axes and a title.
- Plot a histogram of the marks column from your DataFrame and describe the shape of the distribution in one sentence.
- Create a 1x2 subplot showing a line plot of any numeric trend on the left and a scatter plot of two numeric columns on the right.

Assessment Rubric

Criterion	Description
Correctness (60%)	Code runs without errors and produces the expected/correct output
Approach & Efficiency (20%)	Appropriate use of Pythonic / NumPy / Pandas idioms (vectorization over manual loops where relevant)
Code Readability (10%)	Meaningful variable names, comments explaining logic
Presentation (10%)	Clear output labeling, charts properly titled/labeled, organized notebook structure